



УНИВЕРЗИТЕТ У НОВОМ САДУ
ФАКУЛТЕТ ТЕХНИЧКИХ
НАУКА У НОВОМ САДУ



Ана Попарић

Паралелно генерисање фракталних стабала: поређење Python и Rust имплементација

ЗАВРШНИ РАД

Основне академске студије

Нови Сад, 2026

	УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА 21000 НОВИ САД, Трг Доситеја Обрадовића 6	
	КЉУЧНА ДОКУМЕНТАЦИЈСКА ИНФОРМАЦИЈА	

Редни број, РБР :		
Идентификациони број, ИБР :		
Тип документације, ТД :		Монографска документација
Тип записа, ТЗ :		Текстуални штампани материјал
Врста рада, ВР :		Дипломски - бечелор рад
Аутор, АУ :		Ана Попарић
Ментор, МН :		Др Игор Дејановић, редовни професор
Наслов рада, НР :		Паралелно генерисање фракталних стабала: поређење Python и Rust имплементација
Језик публикације, ЈП :		српски/ћирилица
Језик извода, ЈИ :		српски/енглески
Земља публиковања, ЗП :		Република Србија
Уже географско подручје, УГП :		Војводина
Година, ГО :		2026
Издавач, ИЗ :		Ауторски репринт
Место и адреса, МА :		Нови Сад, трг Доситеја Обрадовића 6
Физички опис рада, ФО : (поглавља/страница/ цитата/табела/слика/графика/прилога)		7/63/31/16/16/0/2
Научна област, НО :		Електротехничко и рачунарско инжењерство
Научна дисциплина, НД :		Примењене рачунарске науке и информатика
Предметна одредница/Кључне речи, ПО :		фрактално стабло, паралелизација, Python, Rust, поређење перформанси
УДК		
Чува се, ЧУ :		У библиотеци Факултета техничких наука, Нови Сад
Важна напомена, ВН :		
Извод, ИЗ :		Овај документ представља упутство за писање завршних радова на Факултету техничких наука Универзитета у Новом Саду. У исто време је и шаблон за Turpst.
Датум прихватања теме, ДП :		
Датум одбране, ДО :		01.01.2025
Чланови комисије, КО :	Председник:	Др Петар Петровић, ванредни професор
	Члан:	Др Марко Марковић, доцент
	Члан:	
	Члан, ментор:	Др Игор Дејановић, редовни професор
		Потпис ментора

	UNIVERSITY OF NOVI SAD • FACULTY OF TECHNICAL SCIENCES 21000 NOVI SAD, Trg Dositeja Obradovića 6	
	KEY WORDS DOCUMENTATION	
Accession number, ANO :		
Identification number, INO :		
Document type, DT :	Monographic publication	
Type of record, TR :	Textual printed material	
Contents code, CC :		
Author, AU :	Ana Poparić	
Mentor, MN :	Igor Dejanović, Phd., full professor	
Title, TI :	Parallel Fractal Tree Generation: A Performance Comparison of Python and Rust Implementations	
Language of text, LT :	Serbian	
Language of abstract, LA :	Serbian/English	
Country of publication, CP :	Republic of Serbia	
Locality of publication, LP :	Vojvodina	
Publication year, PY :	2026	
Publisher, PB :	Author's reprint	
Publication place, PP :	Novi Sad, Dositeja Obradovica sq. 6	
Physical description, PD : (chapters/pages/ref./tables/pictures/graphs/appendixes)	7/63/31/16/16/0/2	
Scientific field, SF :	Electrical and Computer Engineering	
Scientific discipline, SD :	Applied computer science and informatics	
Subject/Key words, S/KW :	fractal tree, parallelization, Python, Rust, performance comparison	
UC		
Holding data, HD :	The Library of Faculty of Technical Sciences, Novi Sad	
Note, N :		
Abstract, AB :	This document provides guidelines for writing final theses at the Faculty of Technical Sciences, University of Novi Sad. At the same time, it serves as a Typst template.	
Accepted by the Scientific Board on, ASB :		
Defended on, DE :	01.01.2025	
Defended Board, DB :	President:	Petar Petrović, Phd., assoc. professor
	Member:	Marko Marković, Phd., asist. professor
	Member:	
	Member, Mentor:	Igor Dejanović, Phd., full professor
		Menthor's sign



УНИВЕРЗИТЕТ У НОВОМ САДУ • ФАКУЛТЕТ ТЕХНИЧКИХ НАУКА
21000 НОВИ САД, Трг Доситеја Обрадовића 6

ИЗЈАВА О НЕПОСТОЈАЊУ СУКОБА ИНТЕРЕСА

Изјављујем да нисам у сукобу интереса у односу ментор – кандидат и да нисам члан породице (супружник или ванбрачни партнер, родитељ или усвојитељ, дете или усвојеник), повезано лице (крвни сродник ментора/кандидата у правој линији, односно у побочној линији закључно са другим степеном сродства, као ни физичко лице које се према другим основама и околностима може оправдано сматрати интересно повезаним са ментором или кандидатом), односно да нисам зависан/на од ментора/кандидата, да не постоје околности које би могле да утичу на моју непристрасност, нити да стичем било какве користи или погодности за себе или друго лице било позитивним или негативним исходом, као и да немам приватни интерес који утиче, може да утиче или изгледа као да утиче на однос ментор-кандидат.

У Новом Саду, дана _____

Ментор

Кандидат

Садржај

1	Увод	1
1.1	Шта је Typst?	1
1.2	Припрема текста	3
1.3	Дефинисање појмова и преводи	4
1.4	Стил писања	4
1.5	Граматика	5
1.6	Слике и табеле	5
1.7	Листа за дораде	6
1.8	Цитирање	6
1.9	Предаја рада	8
1.10	Очекивани обим рада	8
2	Фрактално стабло	9
2.1	Фрактали	9
2.2	Рекурзија и бинарно фрактално стабло	10
2.3	Параметри и врсте фракталних стабала	10
2.4	Величина проблема	12
3	Паралелизација	13
3.1	Процесни модел наспрам модела нити	13
3.2	Расподела оптерећења	14
3.3	Амдалов закон	15
3.4	Густафсонов закон	15
4	Имплементација	19
4.1	Секвенцијална имплементација	19
4.2	Стратегија поделе задатака	20
4.3	Паралелна Python имплементација	21
4.4	Паралелна Rust имплементација	22
5	Експериментална евалуација	25
5.1	Тестна платформа и методологија мерења	25
5.2	Јако скалирање	27
5.3	Слабо скалирање	31
6	Дискусија	37
6.1	Апсолутне перформансе: Python наспрам Rust	37
6.2	Јако скалирање	38
6.3	Слабо скалирање	40
6.4	Ограничења примењених модела	41

7 Закључак	43
Списак слика	45
Списак листинга	47
Списак табела	49
Списак коришћених скраћеница	51
Списак коришћених појмова	53
Биографија	55
Литература	57
Грешке у литературним наводима	61
TODOs	63

Овај шаблон је намењен за припрему завршних дипломских и мастер радова на студијском програму за софтверско инжењерство и информационе технологије на Факултету техничких наука. Урађен је употребом [Typst](#) система који је описан у наставку.

1.1 Шта је Typst?

[Typst](#) је систем за припрему, односно аутоматски прелом докумената (енг. *typesetting*), и посебно је погодан за техничке и научне текстове. Омогућава прецизно формирање, математичке формуле, библиографије и сложене структуре. Користи .typ текстуалне фајлове и компајлира их у PDF.

Овакав приступ израде докумената називамо “What You See Is What You Meant” (оно што видиш је оно што си мислио) који је популаран у техничким доменима. На пример, *markdown* који користимо на GitHub-у и другим сајтовим је добар пример овог приступа. Насупрот овом приступу имамо “What You See Is What You Get” (оно што видиш је оно што и добијеш).

Разлике између ова два приступа су следеће:

- **WYSIWYG** (*What You See Is What You Get*) – Офис алати (нпр. Word, Google Docs):
 - Видиш тачно како ће документ изгледати приликом уређивања,
 - Фокус на визуелном изгледу (фонт, боје, размаци),
 - Мање аутоматизације за сложене структуре (референце, садржај).
- **WYSIWYM** (*What You See Is What You Meant*) – Typst:
 - Уноси се *логичка структура* (наслови, секције, математичке формуле), а изглед се одређује касније (преко стилова). Пример:
= Увод
Ово је `_логичка_` ознака наслова, а не визуелни избор фонта.
 - Детаљнија контрола (нпр. табеле, формуле),
 - Аутоматско генерисање садржаја, библиографије, референци у тексту,
 - Аутоматско повезивање референци са делом текста који се референцира,

- Аутоматска нумерација поглавља, секција, слика,
- Аутоматско генерисање индекса слика, табела, листинга,
- Аутоматско распоређивање слика, табела, прелом страна, прелом речи на крају реда итд.
- Могућност скриптовања – нпр. аутоматско пребројавање поглавља, слика, табела и сл. у кључној документацијској информацији код завршних радова,
- Могућност верзионисања — .typ фајлови су обични текст, па се лако прате промене (нпр. са Git).
- Стабилност — боље рукује великим документима (нпр. књиге, дисертације, завршни радови).

Typst је слободан софтвер отвореног кода.

1.1.1 Употреба Typst алата

После инсталације¹ довољно је да из фолдера овог шаблона позовете следећу команду:

```
typst watch zavrzni-rad.typ
```

Ова команда покреће Typst у моду у коме прати промене над фајловима и при сваком промени регенерише PDF фајл.

Отворите PDF фајл `zavrzni-rad.pdf` у неком од прегледача који освежава приказ на сваку промену без губљења позиције (нпр. `Okular`² или `Sumatra PDF`³). Сада у вашем текстуалном едитору мењајте `.typ` фајлове. Видећете да се PDF приказ готово тренутно ажурира.

1.1.2 Фонтови

Овај шаблон користи Liberation фонт па се побрините да вам је исправно инсталиран. Да бисте проверили који фонтови су доступни позовите команду:

```
typst fonts
```

или, да проверите да ли имате Liberation фонт инсталиран:

```
typst fonts | grep -i liberation
```

¹<https://github.com/typst/typst#installation>

²<https://okular.kde.org/>

³<https://www.sumatrapdfreader.org/free-pdf-reader>

1.1.3 Упутство за фајлове у T_upst шаблону

У фајлу `metadata.typ` налазе се метаподаци које је потребно да попуните и који ће се аутоматски применити у свим деловима текста где је то потребно. На пример, ту дефинишете:

- формат стране (a4 или iso-b5),
- наслов рада,
- име аутора,
- ментора,
- апстракт,
- кључне речи,
- чланове комисије за оцену и одбрану
- итд.

Такође, дефинишете и потребне елементе на енглеском језику.

Шаблон је прављен тако да није потребно да се ручно мењају задатак, кључна документацијска информација, насловна страна. Све измене се спроводе кроз варијабле у фајлу `metadata.typ`.

У фајлу `biografija.typ` пишете своју кратку биографију.

У фајлу `literatura.bib` уносите референце у BibTeX формату.

У фолдеру `pglavlja` пишете текст вашег рада. Свако ново поглавље морате да укључите са `#include` директивом у дну `zavrsni-rad.typ` фајла.

У фолдеру `slike` смештате слике у PNG формату.

Остале фајлове не треба да дирате.

Шаблон подржава и A4 и ISO-B5 формат. Прилагођен је двостраној штампи. Пошто се прелом обавља аутоматски формат можете мењати по потреби и није потребно унапред да одлучите да ли ћете користити A4 или ISO-B5.

1.2 Припрема текста

- За припрему рада се користи T_upst + BibTeX за референце по узору на овај шаблон.
- Направите клон (енг. *fork*) овог шаблона и додајте ментора као колаборатора.
- Текст се пише **искључиво на ћирилиц**. За пресловљавање у T_upst можете користити Ћирко [1] који се може интегрисати са разни текстуалним едиторима.
- Текст задатка и чланове комисије ћете добити од ментора.
- Обавезно инсталирајте подршку за српски језик са провером правописа у свом текстуалном едитору.

- Обавезно прочитајте рад у целини бар једном пре слања ментору на читање.
- За мастер рад потребно је написати и рад за Зборник радова ФТН-а. Обим је 4 странице двостубачно. Погледајте пример. Шаблон преузимате са сајта Зборника⁴.

1.3 Дефинисање појмова и преводи

- При првом увођењу одређеног појма, описати га и евентуално навести назив на енглеском.

Пример:

Насупрот језицима опште намене, језици специфични за домен (ЈСД, енг. *Domain-Specific Languages*) представљају рачунарске језике који нуде повећање експресивности кроз употребу концепата и нотација прилагођених и често ограничених на одређени домен проблема.

- Користити уобичајени превод уколико постоји или назив у оригиналу у курзиву (енг. *italics*).
- Акроними настали од енглеских речи се пишу на латиници (нпр. HTML, DSL, CSS, AI).
- *Class diagram* се преводи као "дијаграм класа" а не као "класни дијаграм".

1.4 Стил писања

- Знаци интерпункције:
 - Тачке, зарези, двотачке – пишу се уз претходну реч. Следећа реч почиње после размака.
 - Цртица: пише се са размаком са обе стране.
 - Заграде: пише се уз прву односно последњу реч у загради. Отворена заграда почиње после размака у односу на претходну реч. После затворене заграде такође иде размак.
- Код техничких текстова се обично избегава прво лице једнине и користе пасивни облици где год је то могуће. Нпр. уместо “Ја сам имплементирао” је боље рећи “У раду је приказана имплементација...”.
- Избежавати квалификације:
 - “**Веома** је тешко направити...”
 - “Информациони систем је **страховито** сложен...”
 - “Имплементација је **јак**о компликована...”

⁴<http://www.ftn.uns.ac.rs/ojs/index.php/zbornik>

1.5 Граматика

- “Бисмо, бисте” – пише се спојено.
- “Не” – пише се одвојено ако стоји уз глагол, али спојено са остатком речи ако је у оквиру придева или прилога.

Пример:

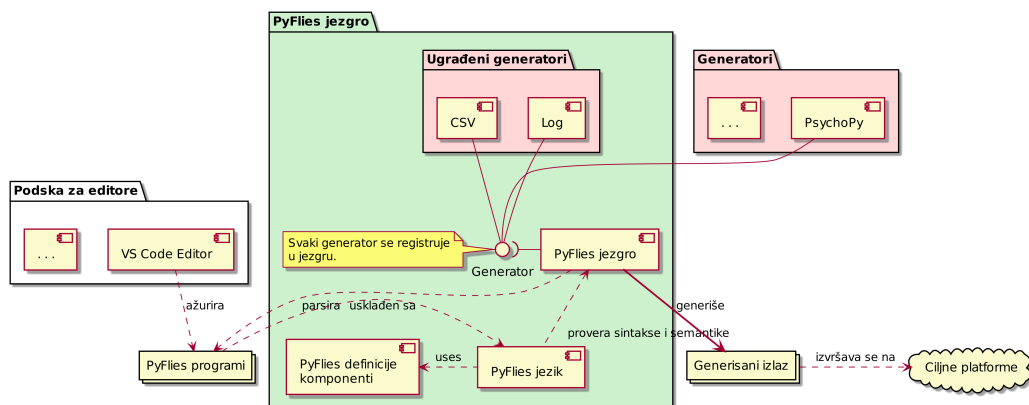
- Придеви и прилози: “недовољно, невидљиво, неразговорно, неизграђен...”.
- Глаголи: “не видим, не могу...”.
- На почетку набрајања ставити двотачку.
- Не користити енглеску конвенцију за писање наслова. У српском језику само прва реч у наслову почиње великим словом.

1.6 Слике и табеле

- За слике које представљају програмски код из едитора избегавати тамне теме. Светле теме боље изгледају у штампи и троше мање тонера. Најбоље је програмски код наводити као листинг у Typst-у јер ће онда бити конзистентно приказан у смислу фонтова и бојења синтаксе.
- Слике и табеле морају имати наслов.
- Свака слика и табела се морају барем једном цитирати у тексту.

Пример:

На слици 1 приказана је PyFlies архитектура која прати стандардне компајлерске архитектуре.



Слика 1: PyFlies архитектура.

Табеле се наводе на следећи начин:

Табела 1: Пример табеле

Податак 1	Податак 2
Вредност 1	Вредност 2

А цитирају у тексту овако [1](#). Можете приметити да ће се списак табела аутоматски креирати на крају на страници [Списак табела](#).

1.7 Листа за дораде

Овај шаблон има подршку за вођење TODO листе која се може користити од стране кандидата док пише текст али и од стране ментора приликом рецензије. Било где у тексту се може навести функција

`#todo`[Нешто што треба да се уради]

Као на пример овде **TODO: Објаснити како се користи todo функција**. И на крају рада ће бити генерисан [списак TODO уноса](#).

1.8 Цитирање

- Цитирајте све коришћене изворе на месту употребе.
- Не преузимајте садржаје дословно (*copy-paste*) већ својим речима препричајте прочитано и пробајте да укрстите са више литературних извора.
- Цитате наводите и у наслову слика које сте преузели. При преузимању слике потребно је да лиценца омогућава употребу у другим радовима.
- Све што не цитирате, а преузели сте из других извора, сматра се плагијатом.
- Цитирање се обавља навођењем броја литературног навода у угластим заградама.
- Цитат је део реченице и стога се наводи пре тачке која завршава реченицу.
- Цитат се у реченици понаша као реч, односно одваја се од околних речи.
- Можете истовремено цитирати више радова. На пример: **[7,14]** или **[1-3,7,12]**.

Примери:

Данас су најпознатије проширена Бакус-Наурова форма (енг. *Extended Backus-Naur form - EBNF*) [13] и аугментована Бакус-Наурова форма.

...

...дефиниција дата је у RFC 5234 документу [14].

- Сви литературни наводи морају имати аутора, наслов, издавача и годину.
- Сви онлајн извори обавезно морају да имају наслов, линк до извора и датум приступа. За онлајн изворе није потребно наводити аутора, издавача и годину уколико нису познати.

Примери:

...

[69] A. Aho, J. Ullman, The theory of parsing, translation, and compiling, Vol. 1 of Series in Automatic Computation, Prentice-Hall, 1972.

...

[72] EBNF: A notation to describe syntax, Online, <https://www.ics.uci.edu/~pattis/misc/ebnf2.pdf>, 2013, приступ: 2021-08-01.

- Радове и књиге можете потражити на Гугл претраживачу за радове⁵. Испод сваке референце имате број цитата (већи број обично значи релевантнији рад), радове који га цитирају (помаже за даљу претрагу) и начин цитирања. Такође, можете директно преузети референцу у BibTeX формату и убацити је у фајл `literatura.bib`.
- Ако можете да бирате увек преферирајте дела који су прошла процес рецензије (радове, књиге), у односу на нерецензиране изворе (веб сајтови, блогови, радне верзије књига и радова, слајдови итд.).

1.8.1 Цитирање употребом BibTeX

BibTeX је текстуални језик/формат за дефинисање референци. Омогућава конзистентно и аутоматско нумерисање и форматирање референци.

Литературне наводе чувате у фајлу `literatura.bib` и у тексту их референцирате са `@ID` где ID представља једнозначни идентификатор референце. На пример, ако у `bib` фајлу имате литературни навод:

```
@book{dejanovic2021c,
  author = {Игор Дејановић},
  title = {Језици специфични за домен},
  publisher = {Факултет техничких наука, Нови Сад},
  year = {2021}
}
```

Листинг 1: Пример BibTeX референце

⁵<https://scholar.google.com/>

У тексту ћете ово дело цитирати са @dejanovic2021c чиме ћете добити овакву референцу [2]. Можете приметити да се у рендерованом PDF-у приказује хиперлинкована референца на литературу на крају рада која је форматирана на прописан начин. Секција са литературом се аутоматски генерише на основу онога што је цитирано у тексту рада. Нумерација се такође аутоматски обавља тако да додавање нових референци, премештање текста итд. неће пореметити исправно бројање и повезивање.

Идентификатор може бити произвољан али је препоручено да буде облика “<презиме аутора><година>”. Ако за аутора имате више радова из исте године додајете суфикс као у претходном примеру.

Претходни блок BibTeX кода на листингу 1 је такође пример како можете у раду користити изворни код. Погледајте изворни код овог шаблона да видите како је наведен код и како се цитира.

Ово је пример цитирања научног рада [3], а ово је пример цитирања онлајн извора [4].

Уколико у BibTeX референцама недостаје неко поље то ће бити пријављено у секцији [Грешке у референцама](#).

Тakoђе, можете се референцирати и на секције и поглавља у оквиру рада. На пример, овако се цитира поглавље 7 (Закључак).

1.9 Предаја рада

- За библиотеку је потребан један тврдо коричени примерак.
- Примерци за комисију — у договору са ментором.
- После одбране, ментор потписује рад за библиотеку. Потребан је и потпис руководиоца студијског програма. Потписан рад кандидат односи у библиотеку ФТН-а као и PDF верзију рада на USB диску.

1.10 Очекивани обим рада

- Дипломски: 40+ страна B5 или 30+ страна A4. Фонт 11.
- Мастер: 60+ B5 или 50+ A4. Фонт 11.

Глава 2

Фрактално стабло

2.1 Фрактали

Термин *фрактал* увео је 1975. године математичар Беноа Манделброт (*Benoît Mandelbrot*) из латинске речи *fractus*, са значењем „сломљен“ или „разломљен“. Фрактал је геометријска фигура која се може поделити на мање делове, при чему сваки од тих делова представља, макар приближно, умањену копију целокупне фигуре [5]. Наведена особина носи назив *само-сличноси* (енг. *self-similarity*) и формално се дефинише на следећи начин:

Геометријска фигура назива се само-сличном уколико у њој постоји тачка са особином да свака њена околина, без обзира на величину, садржи структуру идентичну читавој фигури. Фигура је строго само-слична уколико наведено својство важи у свакој њеној тачки, односно уколико се може раставити на коначан број међусобно дисјунктних подскупова од којих је сваки геометријски подударан са читавом фигуром, уз одговарајући фактор скалирања [6].

Фракталне структуре присутне су свуда у природи — од грађе дрвећа и крвних судова до атмосферских појава попут облака и муња. Класична еуклидска геометрија не може да представи оваква неправилна природна тела, због чега *фрактална геометрија* (енг. *fractal geometry*) представља неопходан математички алат за њихов опис [7]. Пример фракталне структуре у природи приказан је на слици 2.



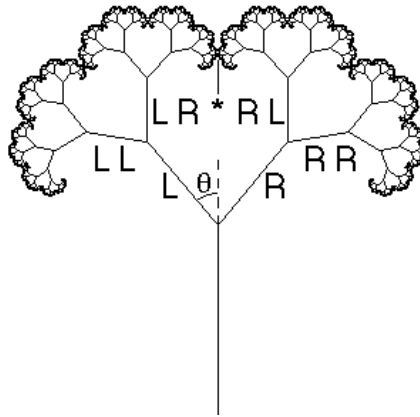
Слика 2: Фрактална структура гранања стабала у природи [8].

2.2 Рекурзија и бинарно фрактално стабло

Кључни рачунарски механизам за генерисање фракталних структура је *рекурзија* (енг. *recursion*) — техника решавања проблема која задатак разлаже на мање задатке исте форме, при чему функција позива сама себе са умањеним потпроблемом, све до достизања *основног случаја* (енг. *base case*) при коме се извршавање зауставља [9].

Бинарно сџабло (енг. *binary tree*) је хијерархијска структура у којој сваки чвор има тачно два потомка — левог и десног [10]. *Бинарно фрактално сџабло* комбинује ову структуру са принципом само-сличности: свака грана рекурзивно се дели на две подгране које су умањене верзије родитеља, а основни случај је достизање минималне дужине гране $l_{\min}$ [11].

Генерисање стабла почиње од дебла (енг. *trunk*) дужине L_0 . Свака грана дели се на две подгране чија је дужина r пута мања од дужине родитељске гране, при чему свака подграна расте под углом θ у односу на правац родитеља. Исти поступак рекурзивно се примењује на свакој новонасталој грани, а рекурзија се зауставља када дужина гране падне испод задатог минималног прага $l_{\min}$, који одговара листовима стабла [11]. Пример генерисаног фракталног стабла приказан је на слици 3.



Слика 3: Пример рачунарски генерисаног бинарног фракталног стабла [12].

2.3 Параметри и врсте фракталних стабала

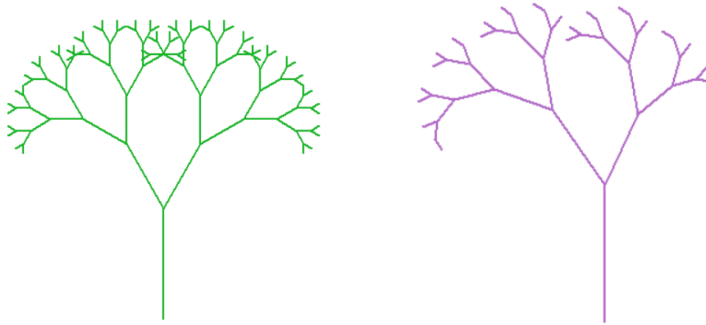
Бинарно фрактално стабло одређено је следећим скупом параметара [11]:

- L_0 — почетна дужина дебла,
- r — фактор смањења дужине гране у односу на родитеља ($0 < r < 1$),
- θ — угао под којим свака подграна расте од правца родитеља,
- $l_{\min}$ — минимална дужина гране испод које рекурзија престаје.

У зависности од тога да ли се исти или различити параметри примењују на леву и десну грану, разликују се два типа стабла.

Симетрично фрактално стабло користи исте параметре за обе подгране — исти фактор смањења и исти угао гранања, при чему обе подгране расту под истим углом у односу на родитеља, само у супротним смеровима [13]. У оквиру овог рада, симетрично стабло је дефинисано параметрима $r_l = r_d = 0.67$ и $\theta_l = \theta_d = 30^\circ$.

Асиметрично фрактално стабло уводи различите параметре за леву и десну грану. У литератури се описују два основна приступа [13]: *Shifted* стабло, код кога се асиметрија постиже померањем једне гране за одређени фактор, и стабло типа *Dragon curve*, код кога се користе два различита угла гранања. У овом раду асиметрија је остварена комбинацијом оба приступа — увођењем различитих фактора смањења ($r_l = 0.67$, $r_d = 0.57$) и различитих углова гранања ($\theta_l = 35^\circ$, $\theta_d = 25^\circ$). Оба типа стабла приказана су на слици 4.



Слика 4: Симетрично (лево) и асиметрично (десно) фрактално стабло.

Избор типа стабла директно утиче на карактеристике паралелног извршавања, па је важно разумети разлику између симетричног и асиметричног случаја.

Симетрично стабло представља математичку идеализацију, јер природни системи готово никад не деле ресурсе равномерно — светлост, гравитација и ветар узрокују да гране расту под различитим угловима и брзинама. Зато асиметрично стабло, са различитим факторима смањења и угловима гранања, ближе одговара реалним биолошким структурама [7].

Код симетричног стабла, идентични фактори смањења ($r_l = r_d$) гарантују да оба подстабла имају исти број грана на свакој дубини, па је расподела посла између леве и десне рекурзивне гране савршено равномерна. Асиметрично стабло нарушава ову равнотежу: лева грана са $r_l = 0.67$ опада спорије него десна са $r_d = 0.57$, па лево подстабло генерише дубљу рекурзију и значајно већи број грана. Ово неравномерно оптерећује процесорске јединице и отежава ефикасну паралелизацију.

2.4 Величина проблема

Максимална дубина рекурзије $d_{\max}$ одређена је условом заустављања: грана постоји све док је њена дужина $l \geq l_{\min}$. Фактор смањења r представља фактор контракције у смислу Барнслијеве теорије [14, стр. 74], па дужина гране на дубини d износи:

$$l_d = L_0 \cdot r^d$$

Из услова заустављања $L_0 \cdot r^d \geq l_{\min}$ непосредно следи максимална дубина рекурзије:

$$d_{\max} = \lfloor \log(l_{\min}/L_0) / \log(r) \rfloor$$

За параметре коришћене у овом раду ($L_0 = 100$, $r = 0.67$, $l_{\min} = 0.01$) добија се:

$$d_{\max} = 22$$

За асиметрично стабло максимална дубина израчунава се одвојено за сваку страну: лева грана са $r_l = 0.67$ достиже дубину $d_l = 22$, а десна са $r_d = 0.57$ достиже дубину $d_r = 17$.

Укупан број генерисаних грана расте експоненцијално са смањењем прага $l_{\min}$. У случају симетричног стабла, када обе гране деле исти фактор скалирања, важи формула за потпуно бинарно стабло [10]:

$$N = 2^{d_{\max}+1} - 1$$

За наведене параметре добија се $N = 8\,388\,607$ грана.

За асиметрично стабло ова формула не важи, јер лева и десна страна достижу праг $l_{\min}$ на различитим дубинама ($d_l = 22$ наспрам $d_r = 17$), па генеришу различит број грана. Укупан број грана асиметричног стабла, одређен приликом генерисања, износи 8 464 173.

Добијени бројеви грана, преко 8 милиона у оба случаја, указују да је генерисање фракталног стабла рачунарски захтеван задатак, те да примена техника паралелног програмирања представља оправдан приступ смањењу времена извршавања.

Глава 3

Паралелизација

3.1 Процесни модел наспрам модела нити

Процес је програм у извршавању коме су потребни системски ресурси: процесорско време, меморија и улазно-излазни уређаји [15, стр. 106].

Меморијски простор процеса подељен је у четири логичке целине [15, стр. 106–107], приказане на слици 5:

- *текстуална секција* — садржи извршни код програма,
- *секција података* — чува глобалне и статичке променљиве,
- *хил* — динамички додељена меморија током извршавања,
- *стек* — локалне променљиве, параметри функција и повратне адресе.



Слика 5: Распоред меморијског простора процеса [15, стр. 107].

Оперативни систем гарантује да је сваки од ових простора потпуно одвојен од меморије других процеса, чиме се спречава да грешка у једном процесу угрози остале [15, стр. 109]. Услед ове изолације, процеси не могу директно приступити меморији једни других, па комуникација међу њима захтева посебне механизме међупроцесне комуникације (енг. *Inter-Process Communication* — IPC).

Поред меморијског простора, сваки процес поседује свој блок контроле процеса (енг. *Process Control Block* — PCB) — структуру у којој оперативни систем чува вредности регистра, бројач програма и информације о ресурсима [15, стр. 109–110].

Иако изолација процеса доноси стабилност, креирање новог процеса носи значајан трошак — додељивање меморијског простора, иницијализација РСВ-а и успостављање IPC канала за комуникацију. Као одговор на овај проблем, савремени оперативни системи уводе концепт нити.

Нити (енг. *thread*) представља основну јединицу коришћења процесора унутар процеса. Нити унутар истог процеса деле извршни код, секцију података и хип, при чему свака нит поседује властити стек и скуп регистара [15, стр. 160–161]. Дељење меморије омогућава ефикасну комуникацију, али истовремено захтева синхронизацију приступа дељеним подацима. Насупрот процесима, нити се креирају са знатно нижим трошком јер деле постојећи меморијски простор процеса [15, стр. 160–162].

Избор између процесног и модела нити зависи од природе задатка [15, стр. 219]:

- *CPU-bound* — задаци који троше процесорско време на израчунавања,
- *I/O-bound* — задаци код којих се претежно чека на улазно-излазне операције.

Како *CPU-bound* задаци не зависе од спољних ресурса, једино уско грло је расположива процесорска снага — због чега паралелно извршавање на више језгара директно резултује убрзањем. Генерисање фракталног стабла типичан је такав задатак — свака рекурзивна грана захтева искључиво аритметичке операције.

3.2 Расподела оптерећења

Паралелно извршавање захтева расподелу посла међу доступним процесорским јединицама. За симетрично стабло оба подстабла имају исту дубину рекурзије и приближно исти број грана — расподела посла је природно равномерна. Асиметрично стабло, међутим, ствара подзадатке различитих величина: лева страна ($r_l = 0.67$, дубина 22) генерише знатно више грана него десна ($r_d = 0.57$, дубина 17). Када се величина подзadataка значајно разликује, количина посла додељена појединим процесорским јединицама постаје неравномерна, што доводи до ситуације у којој неке јединице чекају на завршетак других. Постизање добрих перформанси у таквим случајевима захтева динамичку расподелу оптерећења [16, стр. 2].

Једноставан приступ расподели задатака јесте централни ред задатака (енг. *centralized task queue*) — сви процесори узимају задатке из заједничког реда, чиме настаје уско грло јер сви конкуришу за приступ истом ресурсу. Крађа посла (енг. *work-stealing*) решава овај проблем тако што сваки процесор одржава сопствени ред задатака, а када остане без посла, сам преузима задатке од других заузетих процесора — комуникација се јавља само када је неопходна [16, стр. 3]. Тиме неједнакост у величини подзadataка асиметричног стабла не захтева претходно планирање — оптерећење се динамички расподељује без централног управљања.

3.3 Амдалов закон

Ефикасна расподела посла није једино ограничење паралелног извршавања. Сваки програм садржи секвенцијалне делове који се не могу паралелизовати и као такви одређују горњу границу убрзања без обзира на број доступних процесора. Ову границу описује Амдалов закон.

Амдалов закон даје теоријску горњу границу убрзања програма при додавању процесорских јединица. Ако са f означимо удео програма који се мора извршавати секвенцијално, убрзање при N процесора износи највише:

$$S_{\max} = \frac{1}{f + \frac{1-f}{N}}$$

Када N тежи бесконачности, убрзање конвергира ка $1/f$ — што значи да секвенцијални део програма непропорционално ограничава укупне перформансе, без обзира на број додатих јединица [15, стр. 164]. Овај модел директно се примењује на јако скалирање, где је величина проблема фиксна — секвенцијални удео програма тиме остаје непромењен, па је Амдалов закон валидан оквир за предвиђање убрзања.

У генерисању фракталног стабла, серијски сегмент обухвата секвенцијалну изградњу почетних нивоа стабла пре покретања паралелног извршавања и спајање резултата по његовом завршетку.

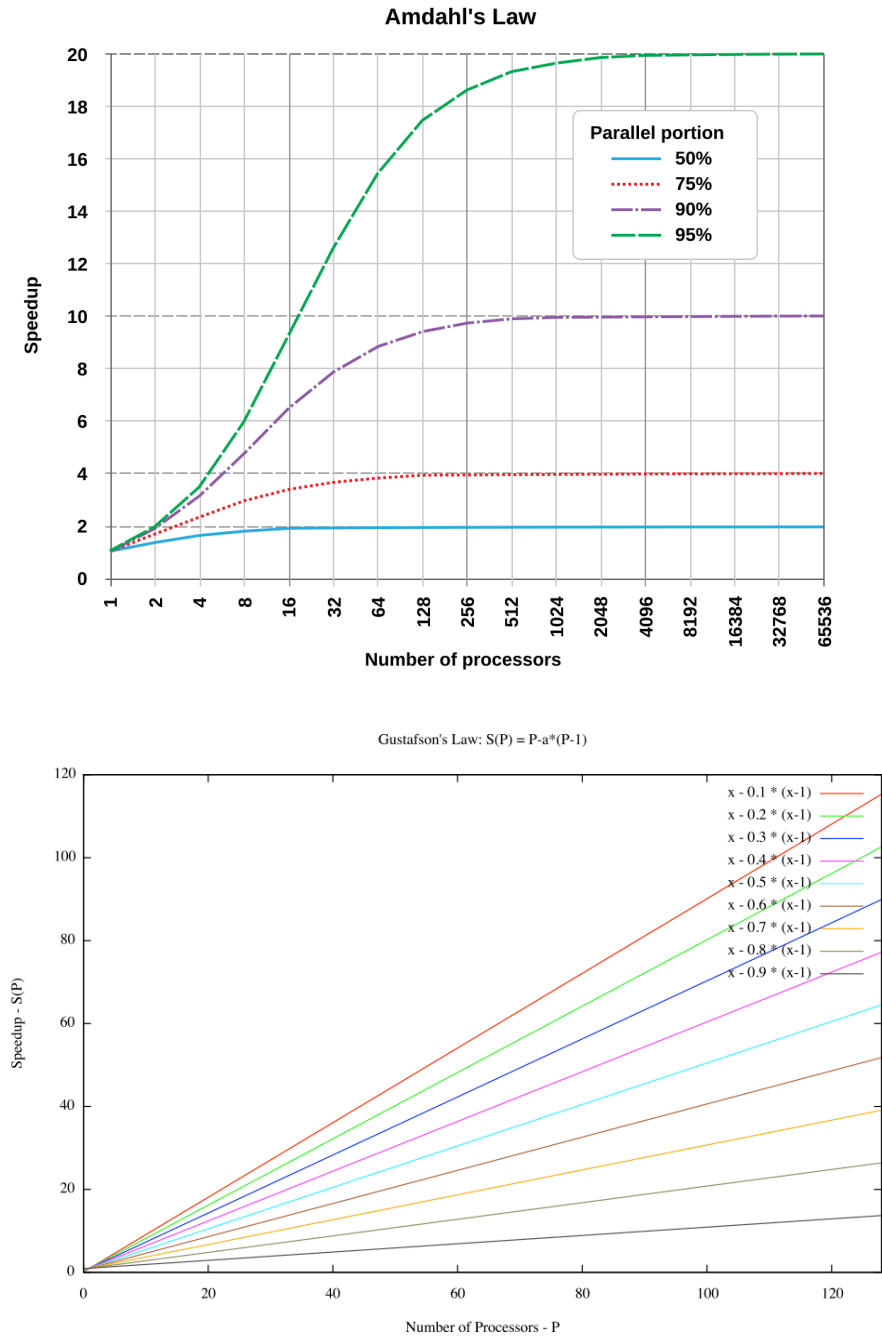
3.4 Густафсонов закон

Густафсонов закон полази од другачије претпоставке него Амдалов: у пракси величина проблема природно расте са бројем расположивих процесора — циљ није решити исти проблем брже, већ решити већи проблем у истом времену [17].

Кључно запажање је да паралелни сегмент програма расте пропорционално са величином проблема, док серијски сегмент остаје приближно константан. Пошто се убрзање не мери над фиксним проблемом већ над проблемом скалираним на N процесора, уводи се појам скалираног убрзања (енг. *scaled speedup*). Ако са f означимо удео времена потрошеног на секвенцијални сегмент, скалирано убрзање при N процесора износи [17]:

$$S_{\text{scaled}} = N + (1 - N) \times f$$

Код Амдаловог закона убрзање расте опадајуће и асимптотски тежи вредности $1/f$, коју никад не достиже. Код Густафсоновог закона убрзање расте линеарно са бројем процесора — без фиксне горње границе, под условом да серијски сегмент остаје константан. Ова разлика приказана је на слици 6.



Слика 6: Амдалов закон (горе) — убрзање асимптотски тежи $1/f$ [18]; Густафсонов закон (доле) — убрзање расте линеарно са бројем процесора [19].

У овом раду, Густафсонов закон примењује се на слабо скалирање (поглавље 5.3): свако процесорско језгро добија проблем исте величине, укупни посао расте са бројем језгара, а серијски сегмент — изградња почетних нивоа стабла и спајање резултата — остаје приближно константан.

Глава 4

Имплементација

У овом поглављу описана је секвенцијална и паралелна имплементација генерисања фракталног стабла у Python-у и Rust-у. Иако Python и Rust верзија следе исту стратегију поделе задатака, два језика се суштински разликују у начину на који управљају меморијом и остварују паралелно извршавање — ове разлике директно одређују избор механизма паралелизације у свакој имплементацији.

Глобална брава интерпретатора (енг. *Global Interpreter Lock* — GIL) је механизам у CPython имплементацији Python-а који у сваком тренутку дозвољава извршавање Python кода само једној нити. GIL постоји јер је управљање меморијом у Python-у засновано на бројању референци — за сваки објекат прати се број показивача на њега, а меморија се ослобађа када тај број падне на нулу. Без заштите, истовремена измена бројача референци довела би до трке у приступу подацима. Пошто сваки процес поседује сопствени интерпретатор и меморијски простор, GIL не представља ограничење на нивоу процеса — вишепроцесно извршавање тако постаје природни избор за постизање паралелности у Python-у [20].

У Rust-у, модел власништва (енг. *ownership*) представља централни механизам за обезбеђивање безбедности меморије и вишенитних програма без потребе за сакупљачем смећа (енг. *garbage collector*). Правилима власништва и позајмљивања, која се проверавају у фази превођења, многе грешке конкурентног програмирања постају грешке компилације, а не грешке током извршавања — чиме се елиминише потреба за механизмима попут GIL-а [21]. Библиотека *Rayon* искоришћава ове гаранције за безбедну паралелизацију преко нити [22].

4.1 Секвенцијална имплементација

Генерисање фракталног стабла засновано је на рекурзивној функцији која у сваком позиву генерише једну грану и рекурзивно обрађује леву и десну подграну. Свака грана одређена је координатама почетне тачке (x_1, y_1) и крајње тачке (x_2, y_2) , уз метаподатак о дубини d на којој је генерисана. Улазни параметри функције су:

- (x, y) — координате почетне тачке гране,
- l — дужина гране,
- α — угао који грана заклапа са хоризонталом.

Крајња тачка израчунава се тригонометријски:

$$x_2 = x + l \cdot \cos(\alpha), \quad y_2 = y + l \cdot \sin(\alpha)$$

Када дужина l падне испод унапред задатог минимума $l_{\min}$, рекурзија се прекида и грана се не генерише. У свим осталим случајевима, након што се текућа грана забележи, функција се позива два пута — једном за леву подграну са углом $\alpha + \theta$, и једном за десну са углом $\alpha - \theta$, при чему се дужина у оба случаја скалира фактором r , а крајња тачка (x_2, y_2) тренутне гране постаје почетна тачка наредног позива.

Симетрично стабло користи исте параметре r и θ за обе подгране, а асиметрично стабло уводи независне параметре — (r_l, θ_l) за леву и (r_d, θ_d) за десну. Rust и Python имплементације деле ову рекурзивну логику, а разликују се у начину представљања и складиштења грана и у моделу паралелизације.

4.2 Стратегија поделе задатака

Будући да свака грана зависи од крајње тачке свог родитеља, алгоритам из секције 4.1 не може се директно паралелизовати. Решење лежи у подели стабла на независна подстабла — стабло се пресеца на дубини задатој параметром `split_depth`, након чега се сваки део може обрађивати независно.

Горњи нивои стабла генеришу се истим рекурзивним обиласком описаним у секцији 4.1, уз једну измену у услову заустављања — обилазак не стаје на минималној дужини гране, већ на дубини `split_depth`. Сваки чвор на тој дубини постаје корен независног подстабла и бележи се као засебан задатак описан координатама, дужином, углом и дужином. Горње гране складиште се одвојено и на крају спајају са паралелно генерисаним гранама.

Вредност `split_depth` одређује се формулом:

$$\text{split_depth} = \lfloor \log_2(N \times 4) \rfloor$$

где је N број расположивих процесорских јединица. Фактор 4 осигурава да се генерише четири пута више задатака него расположивих процесорских јединица — ако би свакој јединици припао тачно један задатак, јединице које заврше пре других би беспослено чекале на преостале. Вишак задатака омогућава да свака слободна јединица одмах преузме следећи задатак, чиме се искоришћеност повећава нарочито у асиметричном стаблу где подстабла нису исте величине. Вредност фактора 4 одређена је на основу емпиријске анализе — при датој хардверској конфигурацији убрзање расте до одређене дубине поделе, а затим опада. За $N = 8$, `split_depth` износи 5 — на тој дубини стабло има $2^5 = 32$ чворова, односно 32 независна задатка за паралелно извршавање.

4.3 Паралелна Python имплементација

Python имплементација остварује паралелизацију на нивоу процеса — библиотека `multiprocessing` покреће независне процесе и тако омогућава стварно паралелно извршавање на вишеструким процесорским јединицама. Комуникација главног процеса са радним процесима одвија се посредством `pickle` серијализације — задаци се пре слања пакују у бајт-ток, а резултати при враћању распакују. Имплементација се одвија у четири фазе.

У првој фази утврђује се број радних процеса N — типично једнак броју расположивих процесорских јединица — и израчунава `split_depth` по формули из секције 4.2. Као полазна тачка рекурзије, дебло стабла генерише се засебно и уписује у листу горњих грана `upper_branches`.

У другој фази примењује се стратегија поделе описана у секцији 4.2 и производе се две листе: `upper_branches` са горњим гранама и `tasks` са параметрима независних подстабала спремних за паралелну обраду.

У трећој фази задаци се дистрибуирају позивом `pool.map` над скупом процеса `Pool(processes=N)`. За сваки задатак позива се секвенцијална функција из секције 4.1 са одговарајућим параметрима. Пре почетка рекурзије, израчунава се укупан број грана у подстаблу и алоцира се `NumPy` низ тачно потребне величине, при чему сваки ред садржи (x_1, y_1, x_2, y_2, d) . За симетрично подстабло број грана одређује се формулом $2^{d_{\max}+1} - 1$ из секције 2.4.

За асиметрично подстабло ова формула не важи јер лева и десна грана имају различите факторе смањења — лева страна достиже $l_{\min}$ на дубини 22, а десна на дубини 17. Свака путања кроз стабло генерише различит број грана у зависности од тога колико пута је скренула лево, а колико десно. Функција `_count_asymmetric` искоришћава ту особину: уместо да броји гране за сваку путању појединачно, она стање рекурзије представља паром (a, b) , где је a број левих, а b број десних скретања од корена. Дужина гране у стању (a, b) износи $L_0 \cdot r_l^a \cdot r_d^b$, тако да је услов заустављања одређен искључиво тим паром — не целом путањом. Многе различите путање кроз стабло завршавају у истом стању (a, b) : на пример, лево-десно и десно-лево обе дају $(1, 1)$. `lru_cache` декоратор осигурава да се вредност за свако стање израчуна само једном и затим кешира.

```
@lru_cache(maxsize=None)
def count(a, b):
    if starting_length * (left_ratio ** a) * (right_ratio ** b) <
    min_length:
        return 0
    return 1 + count(a + 1, b) + count(a, b + 1)
```

Листинг 2: Мемоизована функција за бројање грана асиметричног подстабла

Стабло са преко 8 милиона грана своди се на свега око 459 јединствених стања над којима функција оперише. Рекурзивна функција уписује сваку грану директно на следећу слободну позицију у алоцираном низу и враћа низ грана главном процесу.

У четвртој фази, листа `upper_branches` претвара се у *NumPy* низ, а позивом `np.concatenate` горње гране и резултати свих паралелних процеса спајају се у јединствен низ свих грана стабла.

4.4 Паралелна Rust имплементација

За разлику од Python-а, Rust паралелизација остварује се на нивоу нити, посредством библиотеке *Rayon*. Rust имплементација организована је по истом принципу као Python верзија из секције 4.3 — припрема, подела, паралелно извршавање и спајање резултата. *Rayon* омогућава паралелизам података тако што секвенцијалне итераторе замењује паралелним еквивалентима путем `par_iter()`, при чему логика алгорита остаје непромењена. Захваљујући алгоритму крађе посла, *Rayon* самостално и равномерно расподељује посао међу нитима, без потребе за ручним управљањем [22]. Стратегија поделе из секције 4.2 рекурзивни проблем своди на равну листу независних задатака, над којом се паралелни итератор примењује директно.

У првој фази утврђује се број нити N , израчунава `split_depth` по формули из секције 4.2, и гради се скуп нити са N нити. Дебло стабла уписује се у вектор `upper_branches` као вредност типа `Branch` — Rust структуре са пољима (x_1, y_1, x_2, y_2, d) , еквивалентне реду у *NumPy* низу из секције 4.3.

У другој фази обилазе се горњи нивои стабла и за сваки чвор на дубини `split_depth` бележе се улазни параметри потребни за генерисање одговарајућег подстабла, чиме се формира вектор задатака `tasks`.

Трећа фаза обухвата паралелно извршавање задатака преко скупа нити. За свако подстабло позива се секвенцијална функција из секције 4.1 са одговарајућим параметрима. Пре рекурзије, одређује се укупан број грана у подстаблу и алоцира се вектор типа `Vec<Branch>` тачне величине позивом `Vec::with_capacity`. За симетрична подстабла бројање се врши функцијом `count_branches`, а за асиметрична функцијом `count_branches_asymmetric`, при чему обе примењују рекурзију без мемоизације. Када је исти фактор смањења примењен на обе гране стабла, сви задаци на дубини `split_depth` садрже исти број грана и оптерећење је равномерно распоређено — случај симетричног стабла. У асиметричном стаблу генеришу се задаци неједнаких величина, јер лева и десна подстабла расту различитом брзином и достижу $l_{\min}$ на различитим дубинама. Алгоритам крађе посла, поменут у уводу секције, управо је намењен таквим сценаријима — неуједначена оптерећења расподељују се динамички.

У завршној фази резултати сваке нити сакупљају се у структуру типа `Vec<Vec<Branch>>`. Будући да је за сваку нит резервисан засебан вектор, спајање

резултата у јединствен низ није неопходно — укупан број грана добија се сабирањем дужина појединачних вектора.

Глава 5

Експериментална евалуација

У овом поглављу представљени су резултати мерења перформанси паралелне имплементације генерисања фракталног стабла у Python-у и Rust-у. Мерења обухватају јако и слабо скалирање за симетрично и асиметрично стабло, као и анализу утицаја параметра `split_depth` на ефикасност паралелизације.

5.1 Тестна платформа и методологија мерења

Сва мерења извршена су на систему чија је хардверска конфигурација приказана у табели 2. Будући да је реч о лаптоп процесору са ограниченим капацитетом хлађења, код дуготрајних експеримената може доћи до термалног ограничавања (*throttling*) [23].

Табела 2: Хардверска конфигурација тестног система

Атрибут	Вредност
Процесор	Intel Core i5-1035G1
Физичка / логичка језгра	4 / 8 (Hyper-Threading)
L2 кеш (по језгру)	512 KB
Меморија	8 GB DDR4, single-channel

Верзије коришћених компоненти дате су у табели 3.

Табела 3: Верзије коришћених компоненти

Компонента	Верзија
Оперативни систем	Windows 11 Pro
Python	3.11 (multiprocessing, spawn метода)
Rayon	1.10
rustc / cargo	1.91.0

Параметри фракталног стабла и конфигурација мерења дати су у табели 4.

Табела 4: Параметри фракталног стабла и конфигурација мерења

Параметар	Вредност
Дужина дебла	$l_0 = 100$
Фактор смањења (симетрично)	$r = 0.67$
Леви / десни фактор (асиметрично)	$r_l = 0.67 / r_d = 0.57$
Угао гранања (симетрично)	$\theta = 30^\circ$
Леви / десни угао (асиметрично)	$\theta_l = 35^\circ / \theta_d = 25^\circ$
Минимална дужина — симетрично (јако скалирање)	$l_{\min} = 0.01$
Минимална дужина — асиметрично (јако скалирање)	$l_{\min} = 0.0023$
Број понављања по конфигурацији	10
Број процесорских јединица	$N \in \{1, 2, 4, 8\}$

Вредности минималне дужине гране одабране су тако да оба типа стабла производе приближно исти укупни број грана — симетрично стабло садржи 8 388 607, а асиметрично 8 464 173 гране, чиме је обезбеђена еквивалентна величина проблема при поређењу резултата.

Временски интервал мерења обухвата цело извршавање паралелне функције — укључујући поделу задатака и прикупљање резултата — тако да измерено време садржи и секвенцијални и паралелни део алгорита. Мерење при $N = 1$ представља чисто секвенцијалну референтну вредност, при чему се позива секвенцијална имплементација без *overhead* управљања процесима. За сваку конфигурацију извршено је десет мерења ради смањења утицаја случајних варијација у извршавању. Пре израчунавања просека из скупа мерења искључене су екстремне вредности методом интерквартилног распона (IQR). Стандардна девијација σ израчунава се на основу очишћеног скупа мерења и приказана је у табелама резултата као показатељ поузданости мерења.

За сваку конфигурацију параметра N вредност `split_depth` одређена је формулом из секције 4.2. Табела 5 приказује конкретне вредности `split_depth` и одговарајући број задатака за сваку конфигурацију N коришћену у експериментима.

Табела 5: Вредности `split_depth` по конфигурацији; број задатака износи $2^{\text{split_depth}}$

N	<code>split_depth</code>	Број задатака
2	3	8
4	4	16
8	5	32

5.2 Јако скалирање

Јако скалирање односи се на мерење убрзања програма при повећању броја процесорских јединица уз фиксну величину проблема. Убрзање се израчунава као однос времена извршавања на једној процесорској јединици и времена извршавања на N јединица [24]:

$$S = \frac{T(1)}{T(N)}$$

На основу измереног убрзања S за дато N процењује се паралелна фракција p инверзијом Амдаловог закона [25]:

$$p = \frac{N(S - 1)}{S(N - 1)}$$

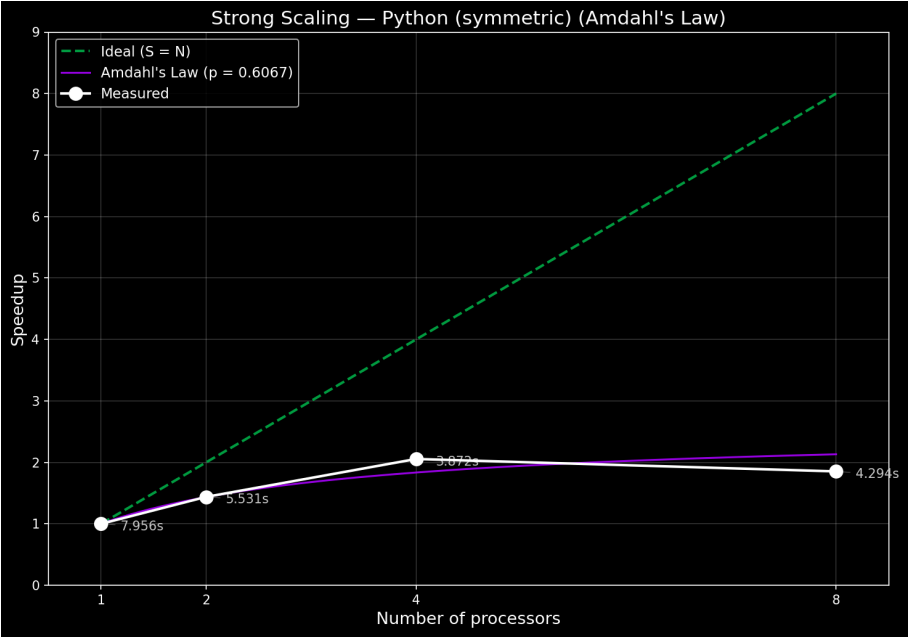
Пошто свако мерење даје различиту процену p , у табелама је приказана средња вредност процена добијених за свако $N > 1$.

5.2.1 Симетрично стабло

Симетрично стабло са $l_{\min} = 0.01$ садржи 8 388 607 грана. Резултати мерења приказани су у табели 6 и табели 7, а криве убрзања на слици 7 и слици 8.

Табела 6: Јако скалирање — симетрично стабло, Python ($p = 0.607$)

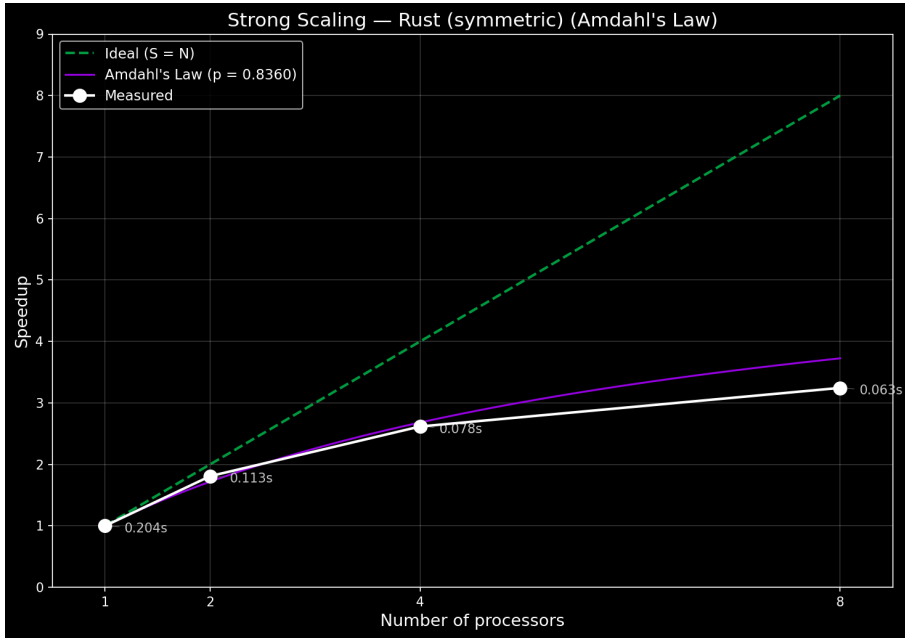
N	$T(N)$ (s)	σ (s)	S	Амдал
1	7.956	0.133	1.000	1.000
2	5.531	0.190	1.438	1.435
4	3.872	0.112	2.055	1.835
8	4.294	0.038	1.853	2.132



Слика 7: Убрзање при јаком скалирању — симетрично стабло, Python

Табела 7: Јако скалирање — симетрично стабло, Rust ($p = 0.836$)

N	$T(N)$ (s)	σ (s)	S	Амдал
1	0.204	0.002	1.000	1.000
2	0.113	0.004	1.808	1.718
4	0.078	0.007	2.616	2.681
8	0.063	0.001	3.244	3.725



Слика 8: Убрзање при јаком скалирању — симетрично стабло, Rust

На основу резултата уочавају се следеће карактеристике:

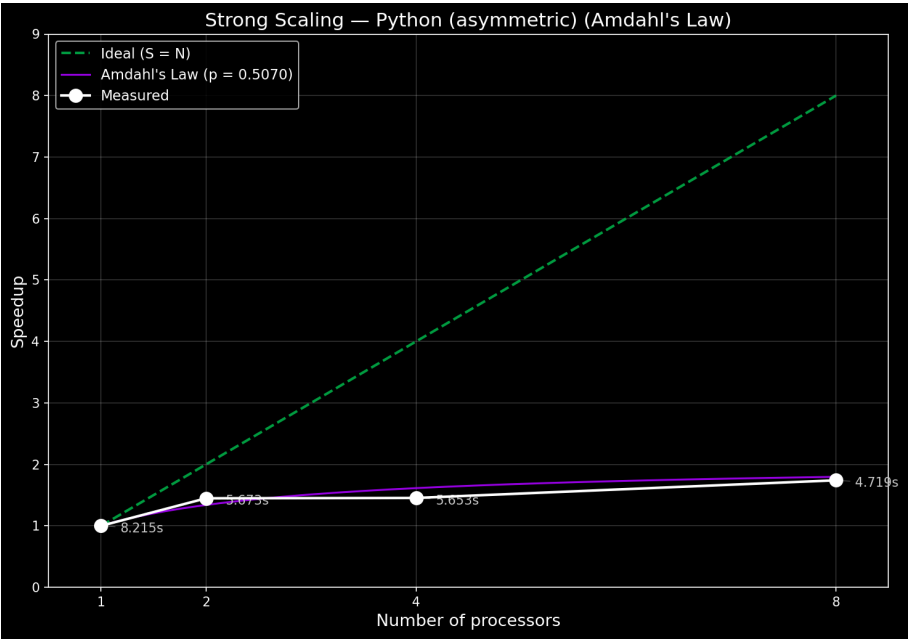
- Python убрзање достиже максимум при $N = 4$, а при $N = 8$ опада. При $N = 4$ измерено убрзање надмашује Амдалову прогнозу, а при $N = 8$ пада испод ње.
- Rust убрзање монотонно расте кроз све конфигурације. При $N = 4$ измерена и предвиђена вредност готово се поклапају, а при $N = 8$ измерено убрзање заостаје за прогнозом, при чему одступање расте са N .

5.2.2 Асиметрично стабло

Асиметрично стабло са $l_{\min} = 0.0023$ садржи 8 464 173 гране. Резултати мерења приказани су у табели 8 и табели 9, а криве убрзања на слици 9 и слици 10.

Табела 8: Јако скалирање — асиметрично стабло, Python ($p = 0.507$)

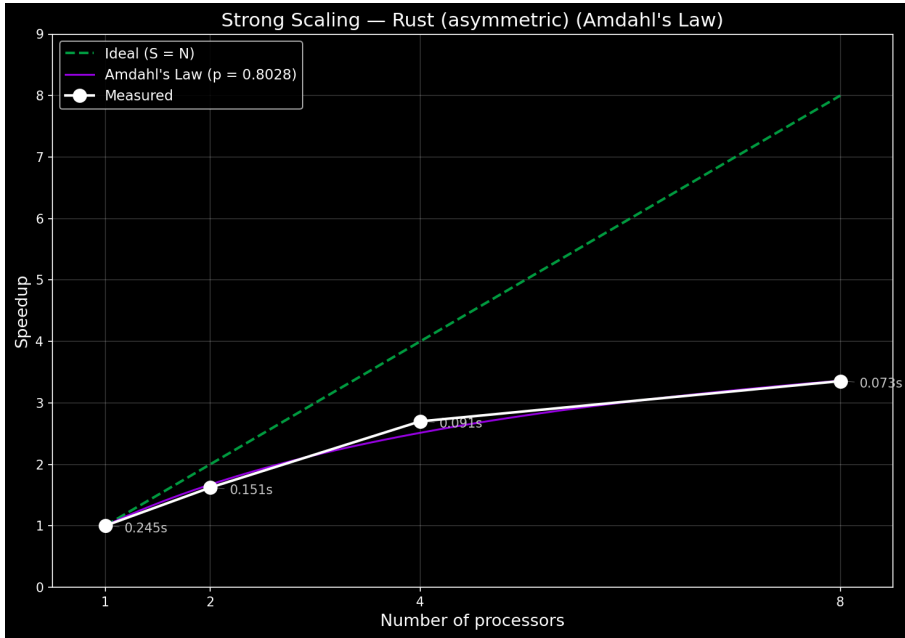
N	$T(N)$ (s)	σ (s)	S	Амдал
1	8.215	0.299	1.000	1.000
2	5.673	0.185	1.448	1.340
4	5.653	0.793	1.453	1.614
8	4.719	0.196	1.741	1.797



Слика 9: Убрзање при јаком скалирању — асиметрично стабло, Python

Табела 9: Јако скалирање — асиметрично стабло, Rust ($p = 0.803$)

N	$T(N)$ (s)	σ (s)	S	Амдал
1	0.245	0.012	1.000	1.000
2	0.151	0.010	1.622	1.671
4	0.091	0.004	2.700	2.513
8	0.073	0.002	3.355	3.361



Слика 10: Убрзање при јаком скалирању — асиметрично стабло, Rust

На основу резултата уочавају се следеће карактеристике:

- Python убрзање монотono расте, али при $N = 4$ готово стагнира. При $N = 2$ измерено убрзање надмашује Амдалову прогнозу, а при вишим вредностима N заостаје за њом.
- Rust убрзање монотono расте кроз све конфигурације. При $N = 2$ измерено убрзање заостаје за прогнозом, при $N = 4$ надмашује прогнозу, а при $N = 8$ готово се савршено поклапа са прогнозом.

5.3 Слабо скалирање

Слабо скалирање односи се на мерење при коме се величина проблема повећава сразмерно броју процесорских јединица — свака јединица задржава приближно исту количину посла. Будући да Густафсон претпоставља константно време извршавања уместо константне величине проблема, природна мера перформанси је количина посла обављеног у том времену. Скалирано убрзање тако представља однос хипотетичког серијског времена за N пута већи проблем и стварног времена паралелног извршавања [17]:

$$S_{\text{scaled}} = \frac{N \cdot T(1)}{T(N)}$$

На основу измереног S_{scaled} за дато N процењује се паралелна фракција p инверзијом Густафсоновог закона [17]:

$$p = \frac{S_{\text{scaled}} - 1}{N - 1}$$

Пошто свако мерење даје различиту процену p , у табелама је приказана средња вредност процена добијених за свако $N > 1$.

Величина проблема регулисана је вредношћу $l_{\min}$, која са растом N опада тако да укупни број грана расте приближно пропорционално N . Вредности $l_{\min}$ одређене су на основу структуре фракталног стабла: смањење $l_{\min}$ за фактор r додаје приближно један ниво гранања, чиме се број грана отприлике удвостручује. Пошто се N удвостручује у сваком кораку ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8$), $l_{\min}$ се множи са $r = 0.67$ за симетрично стабло, односно са $\sqrt{r_l \cdot r_d} = \sqrt{0.67 \cdot 0.57} \approx 0.618$ за асиметрично стабло. Коришћене вредности приказане су у табели 10.

Табела 10: Вредности $l_{\min}$ и број грана по конфигурацији за слабо скалирање

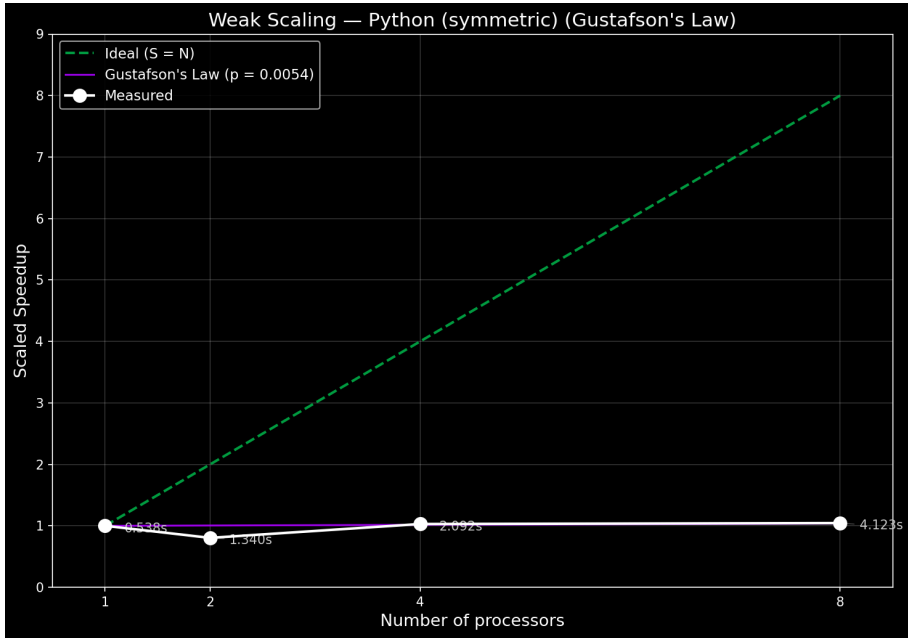
N	$l_{\min}$ (сим.)	Грана (сим.)	$l_{\min}$ (асим.)	Грана (асим.)
1	0.04	1 048 575	0.01	919 442
2	0.0268	2 097 151	0.00618	1 855 434
4	0.017956	4 194 303	0.003819	3 731 355
8	0.012031	8 388 607	0.002360	7 518 053

5.3.1 Симетрично стабло

Резултати мерења приказани су у табели 11 и табели 12, а криве скалираног убрзања на слици 11 и слици 12.

Табела 11: Слабо скалирање — симетрично стабло, Python ($p = 0.005$)

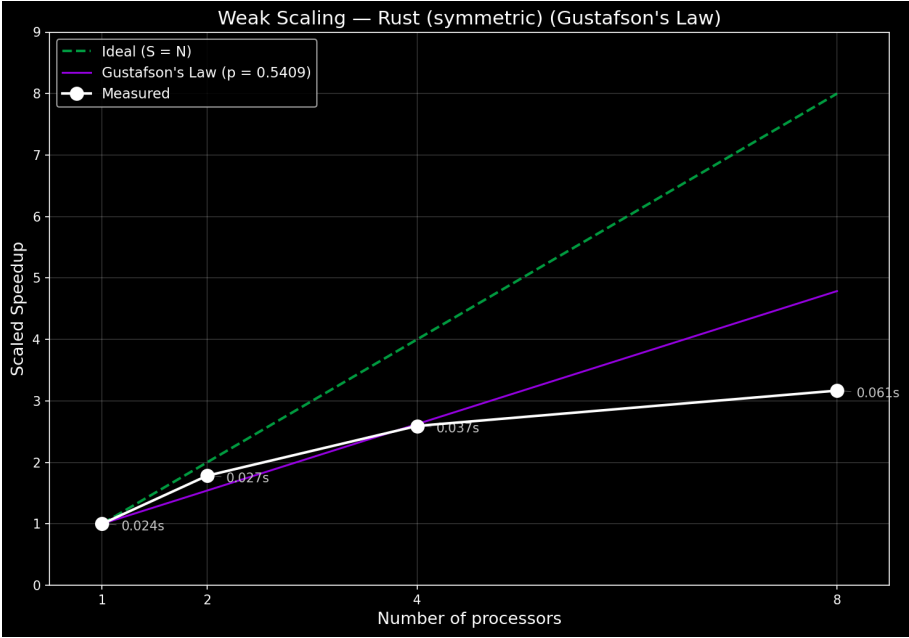
N	$T(N)$ (s)	σ (s)	S_{scaled}	Густафсон
1	0.538	0.008	1.000	1.000
2	1.340	0.031	0.804	1.005
4	2.092	0.080	1.029	1.016
8	4.123	0.151	1.045	1.038



Слика 11: Скалирано убрзање при слабом скалирању — симетрично стабло, Python

Табела 12: Слабо скалирање — симетрично стабло, Rust ($p = 0.541$)

N	$T(N)$ (s)	σ (s)	S_{scaled}	Густафсон
1	0.024	0.001	1.000	1.000
2	0.027	0.001	1.782	1.541
4	0.037	0.005	2.593	2.623
8	0.061	0.003	3.168	4.787



Слика 12: Скалирано убрзање при слабом скалирању — симетрично стабло, Rust

На основу резултата уочавају се следеће карактеристике:

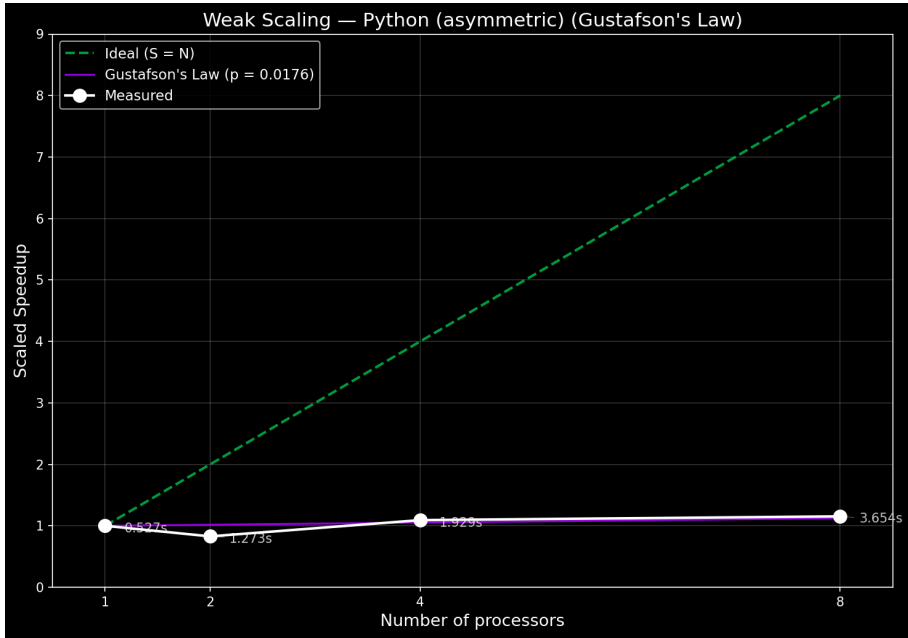
- Python скалирано убрзање при $N = 2$ пада испод 1. При $N = 4$ и $N = 8$ расте изнад 1 и блиско је Густафсоној прогнози.
- Rust скалирано убрзање монотono расте кроз све конфигурације. При $N = 2$ измерена вредност превазилази прогнозу, при $N = 4$ приближно одговара прогнози, а при $N = 8$ значајно заостаје за њом.

5.3.2 Асиметрично стабло

Резултати мерења приказани су у табели 13 и табели 14, а криве скалираног убрзања на слици 13 и слици 14.

Табела 13: Слабо скалирање — асиметрично стабло, Python ($p = 0.018$)

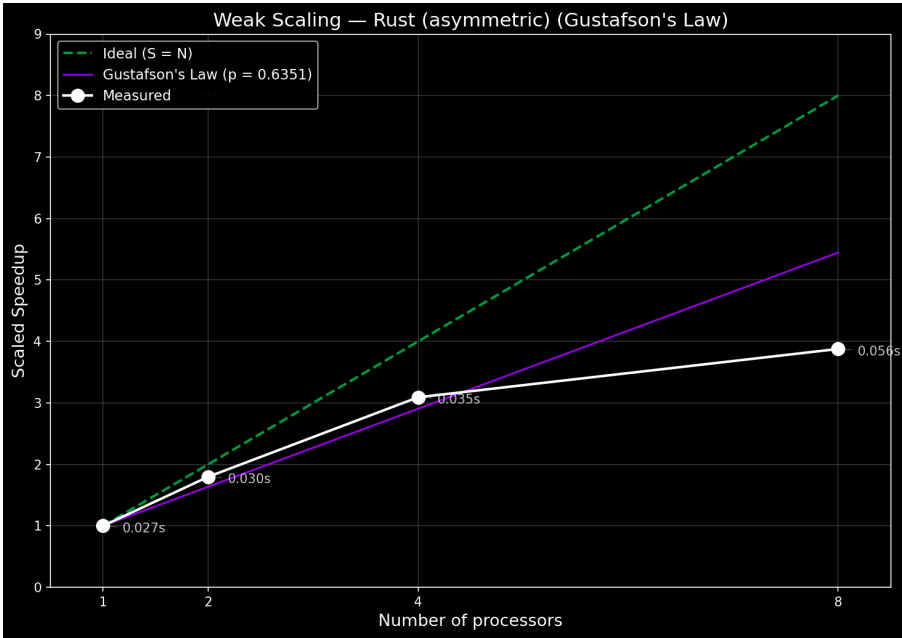
N	$T(N)$ (s)	σ (s)	S_{scaled}	Густафсон
1	0.527	0.007	1.000	1.000
2	1.273	0.014	0.828	1.018
4	1.929	0.091	1.093	1.053
8	3.654	0.045	1.154	1.123



Слика 13: Скалирано убрзање при слабом скалирању — асиметрично стабло, Python

Табела 14: Слабо скалирање — асиметрично стабло, Rust ($p = 0.635$)

N	$T(N)$ (s)	σ (s)	S_{scaled}	Густафсон
1	0.027	0.002	1.000	1.000
2	0.030	0.002	1.797	1.635
4	0.035	0.002	3.092	2.905
8	0.056	0.005	3.878	5.446



Слика 14: Скалирано убрзање при слабом скалирању — асиметрично стабло, Rust
На основу резултата уочавају се следеће карактеристике:

- Python скалирано убрзање при $N = 2$ пада испод 1, слично симетричном случају. При $N = 4$ и $N = 8$ расте изнад 1 и превазилази Густафсонову прогнозу.
- Rust скалирано убрзање монотono расте кроз све конфигурације, достижући веће вредности него у симетричном случају. При $N = 2$ и $N = 4$ измерена вредност превазилази прогнозу, а при $N = 8$ значајно заостаје за њом.

У овом поглављу анализирају се резултати из поглавља 5 у контексту карактеристика коришћених технологија и хардверских ограничења тестне платформе. Циљ је објаснити уочене обрасце скалирања, посебно одступања од теоретских прогноза, кроз анализу механизма паралелизације у Python-у и Rust-у.

Сумарни преглед измерених убрзања при јаком скалирању дат је у табели 15, а при слабом скалирању у табели 16.

Табела 15: Сумарни преглед убрзања S при јаком скалирању

N	Python (сим.)	Rust (сим.)	Python (асим.)	Rust (асим.)
1	1.000	1.000	1.000	1.000
2	1.438	1.808	1.448	1.622
4	2.055	2.616	1.453	2.700
8	1.853	3.244	1.741	3.355

Табела 16: Сумарни преглед скалираног убрзања S_{scaled} при слабом скалирању

N	Python (сим.)	Rust (сим.)	Python (асим.)	Rust (асим.)
1	1.000	1.000	1.000	1.000
2	0.804	1.782	0.828	1.797
4	1.029	2.593	1.093	3.092
8	1.045	3.168	1.154	3.878

6.1 Апсолутне перформансе: Python наспрам Rust

При секвенцијалном извршавању ($N=1$), Rust имплементација генерише симетрично стабло приближно 39 пута брже од Python имплементације (0,204 s наспрам 7,956 s), а асиметрично стабло приближно 34 пута брже (0,245 s наспрам 8,215 s).

Ова разлика је директна последица природе оба језика. Python припада категорији интерпретираних програмских језика, што подразумева да се изворни код може покренути непосредно, без претходног превођења у извршну датотеку зависну од циљне платформе. У оквиру CPython имплементације, изворни код се прво компајлира у бајт-код, након чега виртуелна машина тај бајткод интерпретира и извршава. Поред тога, Python користи динамичку типизацију — провера типова не одвија се пре покретања

програма, него у току извршавања, непосредно пре обављања примитивних операција као што су аритметичке операције или приступ атрибутима објеката. Управљање меморијом у CPython-у заснива се на бројачима референци — сваки објекат прати колико референци на њега постоји у датом тренутку, и аутоматски се ослобађа из меморије када тај број достигне нулу [26]. За разлику од интерпретираних језика попут Python-а, Rust је унапред компајлирани језик (*ahead-of-time compiled*) — целокупан изворни код преводи се у машински извршну датотеку пре покретања програма, без потребе за присуством *runtime* окружења на циљном систему [21]. Генерисање фракталног стабла представља рачунски интензиван задатак (*CPU-bound*), у коме програм не чека на спољне ресурсе попут улазно-излазних операција или мрежне комуникације, већ континуирано користи процесор за извршавање рекурзивних математичких израчунавања. У таквим условима, карактеристике извршног модела програмског језика — као што су трошкови интерпретације бајткода, динамичке провере типова и управљања меморијом — директно утичу на укупно вреМЕ извршавања. Из тог разлога, генерисање фракталног стабла представља погодан задатак за поређење перформанси интерпретираног и компајлираног језика.

Наведена разлика представља важан контекст при тумачењу резултата скалирања. Упркос томе што Python може постићи слично релативно убрзање у односу на сопствену секвенцијалну вредност, у апсолутном смислу остаје знатно спорији од Rust имплементације.

6.2 Јако скалирање

6.2.1 *Overhead* Python процесног управљања

Централни узрок ограниченог скалирања Python имплементације је *overhead* настао при управљању процесима, који делује на два нивоа.

На Windows платформи, Python користи `spawn` методу покретања нових процеса — при сваком позиву `Pool(processes=N)` покреће се N потпуно нових Python интерпретера. За разлику од `fork` методе доступне на Unix системима, која само копира меморијски простор постојећег процеса, `spawn` мора испочетка учитати интерпретер, увести све библиотеке и иницијализовати извршно окружење [26]. Овај процес траје мерљиво дуго и понавља се при сваком мерењу.

Поред тога, комуникација са радним процесима одвија се посредством `pickle` серијализације [26]. Задаци се пре слања пакују у бајт-ток, а резултати — NumPy низови са стотинама хиљада до милиона грана по подстаблу — серијализују се при враћању главном процесу. Укупна количина података која се серијализује расте са N и са величином проблема.

Заједнички ефекат ова два фактор видљив је у конкретним мерењима: при $N = 8$, Python симетрично стабло извршава се за 4.294 s — спорије него при $N = 4$ (3.872 s). Убрзање при $N = 8$ тако пада испод убрзања при $N = 4$, јер при $N = 8$ систем покреће 8 нових Python интерпретера и десеријализује 8 скупова резултата — двоструко више него при $N = 4$ — па трошак комуникације надмашује трошак самог рачунања.

При $N = 4$ измерено убрзање премашује Амдалову прогнозу. Разлог је у томе како је параметар p одређен: усредњавањем преко свих вредности $N > 1$. Мерења при $N = 8$, код којих overhead значајно нарушава скалирање, вуку процену p наниже — тиме потцењујући ефективну паралелну фракцију при мањим вредностима N . Последица је да Амдалова крива за $N = 4$ прогнозира мање убрзање него шта мерење стварно показује.

6.2.2 Rust и ефекат Hyper-Threading-a

Rust имплементација остварује паралелизам нитима посредством библиотеке Rayon. За разлику од Python процеса, нити не захтевају покретање новог интерпретера нити серијализацију података — деле меморијски простор процеса и директно приступају задацима и резултатима [22]. Overhead управљања нитима је тако занемарљив у поређењу са самим рачунањем, па Rust убрзање прати теоретску прогнозу знатно верније него Python.

И поред тога, Rust убрзање при $N = 8$ значајно заостаје за Амдаловом прогнозом: за симетрично стабло измерено је 3.244 наспрам прогнозе 3.725. Узрок лежи у хардверском ограничењу: тестна платформа поседује 4 физичка процесорска језгра са технологијом Hyper-Threading, која свако физичко језгро представља као два логичка [15]. При $N = 8$ свих осам нити распоређује се на свега четири физичка језгра — две нити на истом физичком језгру деле исте извршне јединице и L1/L2 кеш меморију [27, стр. 14]. Hyper-Threading побољшава искоришћеност при задацима са чекањем на меморијске операције, али код CPU-bound задатака као што је генерисање стабла не обезбеђује линеарно повећање рачунске снаге [27, стр. 13]. Резултат је да прелазак са $N = 4$ на $N = 8$ не доноси двоструко убрзање, иако се број логичких јединица удвостручује.

6.2.3 Утицај асиметрије стабла

Асиметрично стабло карактеришу подстабла различитих величина: лева грана са фактором $r_l = 0.67$ расте спорије и генерише дубља подстабла, а десна са $r_d = 0.57$ достиже $l_{\min}$ брже. Задаци на истој дубини `split_depth` тако садрже различит број грана — подстабла са претежно левим гранама имају знатно више рачунања него подстабла са претежно десним.

У Python имплементацији та неједнакост директно узрокује стагнацију убрзања при $N = 4$: `pool.map` расподељује задатке статички и чека на завршетак свих процеса пре

него прикупи резултате [26]. Ако неки процес добије знатно веће подстабло од осталих, он постаје уско грло које задржава цео систем. При $N = 4$ тај ефекат доминира над добити од паралелизма, па је убрзање (1.453) готово идентично убрзању при $N = 2$ (1.448) — додатна два процеса практично не доносе напредак.

Rust имплементација овај проблем решава алгоритмом крађе посла (енг. *work-stealing*): нит која заврши свој задатак одмах преузима следећи задатак из реда нити са највише преосталог посла. Тиме се неједнаки задаци динамички расподељују међу нитима, без беспосленог чекања [22]. Последица је да Rust убрзање при $N = 8$ за асиметрично стабло (3.355) готово савршено одговара Амдаловој прогнози (3.361) — боље поклапање него у симетричном случају (3.244 наспрам 3.725), где је крађа посла мање релевантан јер су сви задаци исте величине.

6.3 Слабо скалирање

Резултати слабог скалирања указују на два различита понашања: Python имплементација готово не скалира, а Rust остварује умерено до добро скалирање ограничено Hyper-Threading ефектом при $N = 8$.

Python скалирано убрзање при $N = 2$ пада испод 1 за обе конфигурације стабла (0.804 симетрично, 0.828 асиметрично). То значи да паралелна верзија са два процеса и двоструко већим проблемом извршава посао спорије него секвенцијална верзија са половичним проблемом. Разлог је *overhead spawn + pickle* серијализације описан у секцији 6.2.1 — при малом проблему са којим слабо скалирање почиње тај *overhead* сачињава велики удео укупног времена, па паралелна верзија ради спорије него секвенцијална. Процењена паралелна фракција практично је нула ($p = 0.005$ симетрично, $p = 0.018$ асиметрично) — Густафсонов модел тумачи ово тако да скоро целокупно извршавање делује серијски, иако је реч о последици *overhead*-а, а не о суштинским серијским деловима алгоритма. При $N = 4$ и $N = 8$ рачунски део постаје довољно велик да *overhead* изгуби релативну важност, па скалирано убрзање расте изнад 1 и приближава се Густафсоновој прогнози.

Rust скалирано убрзање расте монотono за оба типа стабла, са значајном паралелном фракцијом ($p = 0.541$ симетрично, $p = 0.635$ асиметрично). Вредност p за асиметрично стабло је виша него за симетрично при свакој измереној вредности N — супротно очекивању на основу резултата јаког скалирања. Објашњење лежи у улози алгоритма крађе посла: при слабом скалирању, проблем расте и задаци постају неравномернији по величини, па механизам крађе посла активније расподељује посао међу нитима и остварује израженију предност него у симетричном случају где су сви задаци исте величине. Одступање од прогнозе при $N = 8$ (3.168 наспрам 4.787 симетрично, 3.878 наспрам 5.446 асиметрично) директна је последица Hyper-Threading ефекта описаног у секцији 6.2.2 — исто физичко ограничење делује и у режиму слабог скалирања.

6.4 Ограничења примењених модела

Амдалов и Густафсонов закон пружају корисну теоретску основу за квантификацију паралелног потенцијала, али почивају на претпоставкама које у пракси нису испуњене.

Оба модела претпостављају да је паралелна фракција p константна и независна од броја процесорских јединица N . У стварности, overhead управљања процесима расте са N — при сваком додатом процесу долази до новог покретања интерпретера и нове серијализације. Ти фиксни трошкови увећавају ефективну серијску фракцију са сваким коракном, па модел систематски прецењује убрзање при великим вредностима N . Ефекат је нарочито изражен у слабом скалирању, где модел процењује $p \approx 0$ иако алгоритам садржи суштински паралелне делове.

Ниједан модел не узима у обзир Hyper-Threading: претпостављају да свако логичко језгро еквивалентно доприноси рачунској снази. Мерења показују да прелазак са $N = 4$ на $N = 8$ не даје двоструко убрзање — ни у Python-у ни у Rust-у — пошто четири додатна логичка језгра деле физичке извршне јединице са постојећих четири.

Додатно ограничење представља термално пригушивање (енг. *throttling*) лаптоп процесора при дуготрајним оптерећењима. *Throttling* је механизам којим процесор аутоматски снижава своју радну фреквенцију када температура пређе одређени праг [23]. Циљ је да се спречи прегревање и оштећење хардвера. Тестна платформа може смањити тактну фреквенцију да би спречила прегревање, нарочито при $N = 8$ где сва логичка језгра раде на пуном оптерећењу. Ово уноси систематску грешку у мерења за коју Амдалов и Густафсонов модел не пружају корекцију.

У раду је имплементирано и евалуирано паралелно генерисање фракталног стабла у програмским језицима Python и Rust. Стратегија паралелизације заснива се на подели стабла на независна подстабла на дубини `split_depth`, која се потом обрађују паралелно. У Python-у паралелизација је остварена вишепроцесним извршавањем посредством библиотеке `multiprocessing`, а у Rust-у вишенитним извршавањем посредством библиотеке `Rayon`.

При секвенцијалном извршавању Rust је приближно 39 пута бржи за симетрично стабло и приближно 34 пута бржи за асиметрично. Та разлика одражава суштинску разлику у извршним моделима: Python интерпретира бајткод у виртуелној машини, проверава типове динамички у току извршавања и управља меморијом бројачима референци, а Rust преводи целокупни изворни код у машински извршни облик пре покретања, без потребе за *runtime* окружењем.

Евалуација јаког скалирања показала је да Python постиже убрзање до 2.055 (симетрично) и 1.741 (асиметрично) при $N = 8$, при чему убрзање за симетрично стабло при $N = 8$ пада испод убрзања при $N = 4$. Централни узрок је *overhead* управљања процесима: на Windows платформи свако покретање скупа процеса подразумева покретање N нових Python интерпретера (`spawn` метода), а пренос резултата одвија се `pickle` серијализацијом NumPy низова. Тај *overhead* расте са N и при $N = 8$ надмашује корист паралелизма. Rust остварује монотono убрзање до 3.244 (симетрично) и 3.355 (асиметрично) при $N = 8$, јер нити деле меморијски простор и немају трошак серијализације. Ограничавајући фактор Rust имплементације је Hyper-Threading: при $N = 8$ свих осам нити расподељује се на четири физичка процесорска језгра, па прелазак са $N = 4$ на $N = 8$ не доноси двоструко убрзање.

Резултати слабог скалирања потврђују исти образац. Python скалирано убрзање пада испод 1 при $N = 2$ — паралелна верзија са двоструко већим проблемом ради спорије него секвенцијална верзија са половичним проблемом — јер *overhead* при малом проблему надмашује корист паралелизма. Густафсонов модел тумачи ово тако да је процењена паралелна фракција практично нула, иако је реч о последици *overhead*-а, а не о суштинским серијским деловима алгоритма. Rust остварује монотono растуће скалирано убрзање до 3.168 (симетрично) и 3.878 (асиметрично) при $N = 8$, ограничено само Hyper-Threading ефектом.

Асиметрично стабло показује различит утицај на два језика. У Python имплементацији неједнаки задаци погоршавају убрзање при $N = 4$ услед статичке расподеле у `pool.map` — убрзање готово стагнира у поређењу са $N = 2$. У Rust имплементацији алгоритам крађе посла (енг. *work-stealing*) из библиотеке *Rayon* динамички расподељује неједнаке задатке и тиме остварује израженију предност него у симетричном случају.

Примењени Амдалов и Густафсонов модел пружили су корисну теоретску основу за квантификацију паралелног потенцијала, али почивају на претпоставци да је паралелна фракција константна и независна од броја процесорских јединица. У пракси, *overhead* у Python имплементацији расте са N , а ниједан модел не узима у обзир Hyper-Threading ни термално пригушивање лаптоп процесора при дуготрајним оптерећењима. Последица је да модели систематски прецењују убрзање при $N = 8$, нарочито за Python.

Могући правци даљег истраживања укључују:

- замену `pickle` серијализације дељеном меморијом посредством `multiprocessing.shared_memory`, чиме би се елиминисао *overhead* преноса резултата у Python имплементацији;
- евалуацију на системима са `fork` методом покретања процеса (Linux), ради искључивања *overhead*-а покретања интерпретера и издавања утицаја серијализације;
- евалуацију на платформи са вишим бројем физичких процесорских језгара без Hyper-Threading-a, ради провере Амдалових и Густафсонових прогноза при $N > 4$;
- анализу утицаја параметра `split_depth` на убрзање у функцији броја процесорских јединица за различите величине проблема.

Списак слика

Слика 1	PyFlies архитектура.	5
Слика 2	Фрактална структура гранања стабала у природи [8].	9
Слика 3	Пример рачунарски генерисаног бинарног фракталног стабла [12].	10
Слика 4	Симетрично (лево) и асиметрично (десно) фрактално стабло.	11
Слика 5	Распоред меморијског простора процеса [15, стр. 107].	13
Слика 6	Амдалов закон (горе) — убрзање асимптотски тежи $1/f$ [18]; Густафсонов закон (доле) — убрзање расте линеарно са бројем процесора [19].	16
Слика 7	Убрзање при јаком скалирању — симетрично стабло, Python	28
Слика 8	Убрзање при јаком скалирању — симетрично стабло, Rust	29
Слика 9	Убрзање при јаком скалирању — асиметрично стабло, Python	30
Слика 10	Убрзање при јаком скалирању — асиметрично стабло, Rust	31
Слика 11	Скалирано убрзање при слабом скалирању — симетрично стабло, Python	33
Слика 12	Скалирано убрзање при слабом скалирању — симетрично стабло, Rust .	34
Слика 13	Скалирано убрзање при слабом скалирању — асиметрично стабло, Python	35
Слика 14	Скалирано убрзање при слабом скалирању — асиметрично стабло, Rust	36

Списак листинга

Листинг 1	Пример BibTeX референце	7
Листинг 2	Мемоизована функција за бројање грана асиметричног подстабла	21

Списак табела

Табела 1	Пример табеле	6
Табела 2	Хардверска конфигурација тестног система	25
Табела 3	Верзије коришћених компоненти	25
Табела 4	Параметри фракталног стабла и конфигурација мерења	26
Табела 5	Вредности <code>split_depth</code> по конфигурацији; број задатака износи $2^{\text{split_depth}}$	26
Табела 6	Јако скалирање — симетрично стабло, Python ($p = 0.607$)	27
Табела 7	Јако скалирање — симетрично стабло, Rust ($p = 0.836$)	28
Табела 8	Јако скалирање — асиметрично стабло, Python ($p = 0.507$)	29
Табела 9	Јако скалирање — асиметрично стабло, Rust ($p = 0.803$)	30
Табела 10	Вредности $l_{\min}$ и број грана по конфигурацији за слабо скалирање	32
Табела 11	Слабо скалирање — симетрично стабло, Python ($p = 0.005$)	32
Табела 12	Слабо скалирање — симетрично стабло, Rust ($p = 0.541$)	33
Табела 13	Слабо скалирање — асиметрично стабло, Python ($p = 0.018$)	34
Табела 14	Слабо скалирање — асиметрично стабло, Rust ($p = 0.635$)	35
Табела 15	Сумарни преглед убрзања S при јаком скалирању	37
Табела 16	Сумарни преглед скалираног убрзања S_{scaled} при слабом скалирању . . .	37

Списак коришћених скраћеница

Скраћеница	Опис
API	Application Programming Interface (апликациони програмски интерфејс)
AWS	Amazon Web Services (Амазон веб сервиси)
CI/CD	Continuous Integration / Continuous Delivery (континуирана интеграција / континуирана испорука)
CORS	Cross-Origin Resource Sharing (размена ресурса између извора и дестинације различитог порекла)
CSS	Cascading Style Sheets (језик за описивање стилова)
DOM	Document Object Model (објектни модел документа)
DTO	Data Transfer Object (објекат за пренос података)
HTTP	HyperText Transfer Protocol (протокол за пренос хипертекста)
JSON	JavaScript Object Notation (формат за размену података)
JWT	JSON Web Token (сигурносни токен заснован на JSON формату)
RLS	Row-Level Security (сигурност на нивоу реда)
REST	Representational State Transfer (скуп правила за комуникацију између клијента и сервера)
RPC	Remote Procedure Call (позив удаљене процедуре)
SQL	Structured Query Language (структурирани упитни језик)
TLS	Transport Layer Security (безбедност транспортног слоја)
UML	Unified Modeling Language (језик за моделовање дијаграма)
URL	Uniform Resource Locator (јединствени идентификатор и локатор ресурса)
UI	User Interface (кориснички интерфејс)
UUID	Universally Unique Identifier (универзално јединствени идентификатор)
WAL	Write-Ahead Logging (записивање операција унапред)

Списак коришћених појмова

Појам	Објашњење
Асинхрони рад	Рад који се изводи независно од главног тока извршавања, омогућавајући наставак других операција без чекања на његов завршетак.
Bucket (S3)	Логичка јединица за складиштење у AWS S3 сервису, која организује фајлове у облаку.
Read-Only	Режим рада у коме су подаци само за читање, без могућности измене.
Cross-platform	Способност софтвера да се извршава на више различитих оперативних система из истог кода.
Connection pool	Механизам за управљање и поновну употребу веза са базом података како би се побољшале перформансе апликације.

Биографија

Овде написати своју кратку биографију.

Литература

- [1] И. Дејановић, „Ћирко - ћирилично-латинични конвертор“. Приступљено: 08. Август 2025. [На Интернету]. Доступно на <https://github.com/igordejanovic/cirko>
- [2] И. Дејановић, *Језици специфични за домен*. Факултет техничких наука, Нови Сад, 2021.
- [3] I. Dejanović, R. Vadera, G. Milosavljević, и Ж. Vuković, „TextX: A Python tool for Domain-Specific Languages implementation“, *Knowledge-Based Systems*, том 115, стр. 1–4, 2017, doi: [10.1016/j.knosys.2016.10.023](https://doi.org/10.1016/j.knosys.2016.10.023).
- [4] I. Dejanović, „A Domain-Specific Language (DSL) for designing experiments in psychology“. [На Интернету]. Доступно на <https://github.com/pyflies/pyflies/>
- [5] „Фрактал“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на <https://sr.wikipedia.org/wiki/%D0%A4%D1%80%D0%B0%D0%BA%D1%82%D0%B0%D0%BB>
- [6] B. Mellor, „Fractals: Self-Similarity and Fractal Dimension“. Приступљено: 15. Јуни 2026. [На Интернету]. Доступно на <https://blakemellor.lmu.build/courses/Symmetry/FractalDimension-Spring2013.pdf>
- [7] H. Gupta, „Fractals in Nature: A Mathematical Perspective“, *International Journal for Research Publication and Seminars*, том 16, изд. 1, стр. 581–584, 2025, doi: [10.36676/jrps.v16.i1.322](https://doi.org/10.36676/jrps.v16.i1.322).
- [8] M. Sabroe, „Fractals Tree on Knudshoved Odde“. Приступљено: 15. Јуни 2026. [На Интернету]. Доступно на https://commons.wikimedia.org/wiki/File:Fractals_Tree_on_Knudshoved_Odde_-_panoramio.jpg
- [9] „Fractals and Recursive Graphics“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на https://web.stanford.edu/class/archive/cs/cs106b/cs106b.1208/lectures/fractals/Lecture10_Slides.pdf
- [10] „Binary tree“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на https://en.wikipedia.org/wiki/Binary_tree
- [11] „Fractal Trees – Definitions“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на <https://www.math.union.edu/research/fractaltrees/FractalTreesDefs.html>
- [12] „Fractal Trees – Definitions“. Приступљено: 15. Јуни 2026. [На Интернету]. Доступно на <https://www.math.union.edu/research/fractaltrees/FractalTreesDefs.html>

-
- [13] M. Anderson и others, „Fractal Trees“. Приступљено: 14. Јуни 2026. [На Интернету]. Доступно на <https://www.ithaca.edu/sites/default/files/2021-11/fractal-trees-poster.pdf>
- [14] M. F. Barnsley, *Fractals Everywhere*, 2nd изд. Academic Press, 1993.
- [15] A. Silberschatz, P. B. Galvin, и G. Gagne, *Operating System Concepts*, 10th изд. Wiley, 2018.
- [16] J. Dinan, S. Olivier, G. Sabin, J. Prins, P. Sadayappan, и C.-W. Tseng, „Dynamic Load Balancing of Unbalanced Computations Using Message Passing“, у *Workshop on Performance Modeling, Evaluation, and Optimization of Parallel and Distributed Systems (PMEO-PDS)*, IEEE, 2007. Приступљено: 17. Јуни 2026. [На Интернету]. Доступно на <https://www.cs.unc.edu/~olivier/pmeo07.pdf>
- [17] J. L. Gustafson, „Reevaluating Amdahl's Law“, *Communications of the ACM*, том 31, изд. 5, стр. 532–533, 1988.
- [18] Daniels220, „Amdahl's Law“. Приступљено: 18. Јуни 2026. [На Интернету]. Доступно на <https://commons.wikimedia.org/wiki/File:AmdahlsLaw.svg>
- [19] „Gustafson's Law“. Приступљено: 18. Јуни 2026. [На Интернету]. Доступно на <https://commons.wikimedia.org/wiki/File:Gustafson.png>
- [20] A. Ajitsaria, „What Is the Python Global Interpreter Lock (GIL)?“. Приступљено: 22. Јуни 2026. [На Интернету]. Доступно на <https://realpython.com/python-gil/>
- [21] S. Klabnik и C. Nichols, „The Rust Programming Language“. Приступљено: 22. Јуни 2026. [На Интернету]. Доступно на <https://doc.rust-lang.org/book/>
- [22] N. Matsakis и J. Stone, „Rayon: A data parallelism library for Rust“. Приступљено: 19. Јуни 2026. [На Интернету]. Доступно на <https://github.com/rayon-rs/rayon>
- [23] Intel Corporation, „Information about Temperature for Intel® Processors“. Приступљено: 25. Јуни 2026. [На Интернету]. Доступно на <https://www.intel.com/content/www/us/en/support/articles/000005597/processors.html>
- [24] E. Khomenko и others, „Mancha3D Code: Multi-Purpose Advanced Non-Ideal MHD Code for High Resolution Simulations in Astrophysics“, *arXiv preprint arXiv:2312.04179*, 2024, Приступљено: 22. Јуни 2026. [На Интернету]. Доступно на <https://arxiv.org/pdf/2312.04179>
- [25] L. Yavits, A. Morad, и R. Ginosar, „The Effect of Communication and Synchronization on Amdahl's Law in Multicore Systems“. Приступљено: 18. Јуни 2026. [На Интернету]. Доступно на <https://arxiv.org/pdf/1306.3302>

- [26] Python Software Foundation, „Python 3 Documentation“. [На Интернету]. Доступно на <https://docs.python.org/>
- [27] Intel Corporation, „Intel® Hyper-Threading Technology Technical User's Guide“. Јануар 2003.

Грешке у литературним наводима

Овде се налази списак грешака у литературним наводима које морате исправити у `literatura.bib` фајлу.

1. Референци `binary-tree-wiki` недостаје поље `year`
2. Референци `fractal-tree-img` недостаје поље `year`
3. Референци `fractal-wiki-sr` недостаје поље `year`
4. Референци `gustafson-law-img` недостаје поље `year`
5. Референци `intel2024thermal throttling` недостаје поље `year`
6. Референци `math-union-fractal` недостаје поље `year`
7. Референци `pyflies` недостаје поље `urldate`
8. Референци `stanford-fractals` недостаје поље `year`

TODOs

1. Објаснити како се користи `todo` функција.